

App Development Project Report

1. Student Details

Name: Samar Ahmad

Roll Number: BS23F3001032

Email: 23f3001032@ds.study.iitm.ac.in

About Me:

I am a student at IIT Madras BS Degree Program with a strong interest in web application development, backend systems, and database-driven technologies. I enjoy building practical, user-focused applications that solve real-world problems. This project allowed me to apply concepts from Flask, relational databases, and frontend templating to build a complete campus placement management system from scratch.

2. Project Details

Project Title: Trekking Management Application (TMA) – V2

Description

The Trekking Management Application is a full-stack web application designed to digitally manage trekking activities, trek staff, and participants. It provides separate functionality for **Admin, Trek Staff, and Trekker/User** roles.

The application reduces dependency on manual spreadsheets and communication by providing centralised trek management, participant registration, booking management, slot tracking, and reporting.

Key Features

Admin

- Manage treks and trekking staff.
- Manage registered trekkers.
- Assign staff to treks.
- View bookings and reports.
- Activate/blacklist users and staff.

Trek Staff

- View assigned treks.
- Manage trek information and available slots.
- View participants.
- Mark completed treks.

Trekker

- Register and log in.
- Browse and search available treks.
- Book and cancel treks.
- View booking history.
- Manage profile.
- Export booking history.

The application also implements **JWT authentication, role-based authorisation, Redis caching, and Celery background tasks.**

3. AI/LLM Declaration

AI/LLM tools were used as an assistive tool during the development of this project.

They were used for:

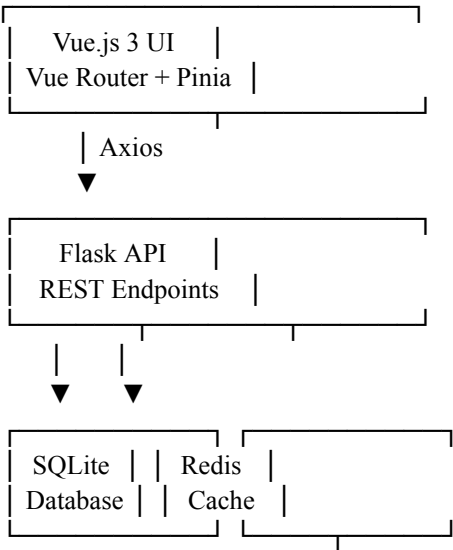
- Understanding and refining requirements.
- Debugging and troubleshooting.
- Improving code structure and documentation.
- Reviewing API and database design.
- Preparing project documentation.

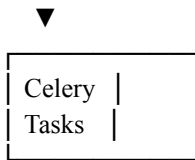
The implementation, integration, testing, and final project decisions were reviewed during development. AI/LLM assistance was used to support the development process and understanding of the project rather than as a replacement for the developer's work.

4. Technologies and Frameworks Used

Component	Technology	Purpose	Category
Frontend	Vue.js 3	User interface and client-side logic	Frontend
Build Tool	Vite	Fast development and building of assets	Development
Backend	Flask	Server-side logic and API management	Backend
API	REST API	Communication between Frontend and Backend	Interface
Database	SQLite	Relational data storage	Storage
ORM	Flask-SQLAlchemy	Object-relational mapping for DB queries	Storage
Authentication	JWT	Secure token-based user authentication	Security
State Management	Pinia	Reactive state management in Frontend	Frontend
Caching	Redis	Data caching to improve response times	Performance
Background Tasks	Celery	Handling asynchronous and scheduled jobs	Backend

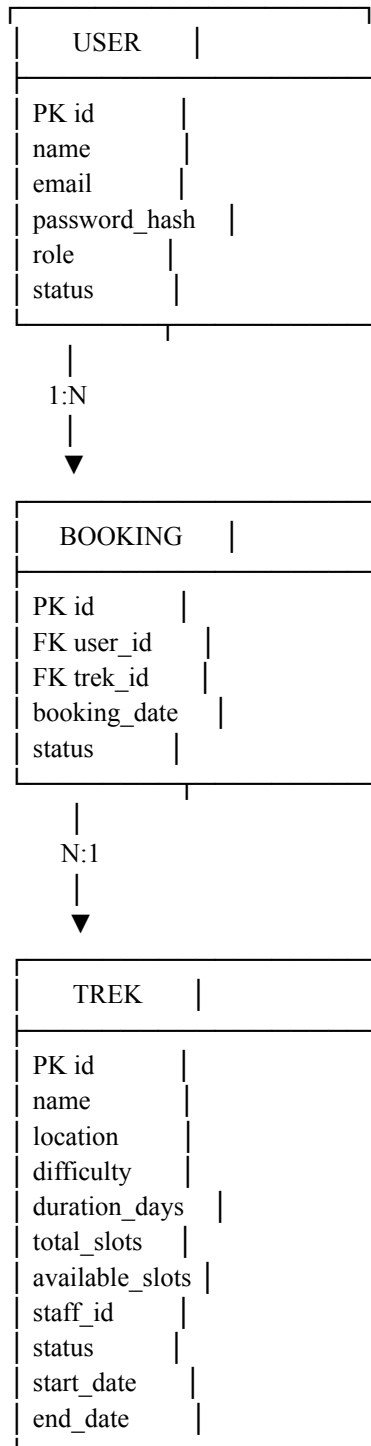
Architecture





5. Database Schema / ER Diagram

The main entities used in the application are **User**, **Trek**, and **Booking**.



Important Relationships

- A **User** can have multiple bookings.
- A **Trek** can have multiple bookings.
- A staff member can be assigned to multiple treks.
- Each booking belongs to one user and one trek.
- The User **role** determines whether the account is Admin, Staff, or Trekker.

6. API Resource Endpoints

Role	Method	Endpoint	Description
Auth	POST	/api/auth/register	Register trekker
Auth	POST	/api/auth/login	Login
Admin	GET	/api/admin/dashboard	Dashboard statistics
Admin	GET	/api/admin/treks	List treks
Admin	POST	/api/admin/treks	Create trek
Admin	PUT	/api/admin/treks/<id>	Update trek
Admin	DELETE	/api/admin/treks/<id>	Delete trek
Admin	GET	/api/admin/staff	List staff
Admin	GET	/api/admin/users	List users
Admin	GET	/api/admin/bookings	View bookings
Admin	GET	/api/admin/reports/summary	Generate summary
Trekker	GET	/api/user/treks	Browse treks
Trekker	POST	/api/user/treks/<id>/book	Book trek
Trekker	PUT	/api/user/bookings/<id>/cancel	Cancel booking
Trekker	GET	/api/user/bookings	Current bookings
Trekker	GET	/api/user/history	Booking history
Trekker	GET	/api/user/profile	View profile
Trekker	PUT	/api/user/profile	Update profile
Trekker	POST	/api/user/export-history	Export history
Staff	GET	/api/staff/dashboard	Staff dashboard
Staff	GET	/api/staff/treks	Assigned treks
Staff	PUT	/api/staff/treks/<id>	Update trek
Staff	PUT	/api/staff/treks/<id>/complete	Complete trek
Staff	GET	/api/staff/treks/<id>/participants	View participants

7. Architecture and Features

Authentication & Authorization

The application uses **JWT-based authentication**. Role-based authorisation restricts Admin, Staff, and Trekker functionality to their respective users.

Booking Management

Before creating a booking, the system checks:

- Trek availability.
- Available slots.
- Existing active booking.

After successful booking, the available slot count is decreased. Cancellation increases the available slot count.

Redis Caching

Redis is used to cache frequently accessed trek listings, improving response time and reducing unnecessary database queries.

Celery Background Tasks

Celery handles operations that do not need to block the main API, including:

- Daily trek reminders.
- Monthly report generation.
- Booking history CSV generation.

Conclusion

The Trekking Management Application provides a centralised platform for managing treks, staff, participants, bookings, and reports. The project demonstrates practical knowledge of **full-stack development, REST APIs, database design, authentication, role-based access control, caching, and asynchronous task processing.**

8. Video Presentation

Drive Link:

 Screen Recording 2026-08-09 at 10.46.19.mov